Assignment 3 | Bridge Pattern

Astana IT University | School of Software Engineering
ShP-2216 Software Design Patterns | 2026-2027
Student: Sabyr Aslan **Group:** SE-2528 **Topic:** A - Drawing

Repository: https://github.com/asikxz008/Assigment3-sdp-Aslan
Base commit: `91f14595faf97a7e6e1086b431af3445ab93eda2`
Submitted source commit: `807f2ed62e4d7a3943703d7e21214f50b375d3a8`

1. Design and pattern choice

The independent dimensions are **shape type** (Circle or Square) and **rendering style** (vector, raster or ASCII). A Circle keeps its radius; a Square keeps its side. Each shape also retains an ID. The Renderer interface handles only the rendering description. Bridge connects these dimensions through the private `Shape.renderer` reference, supplied by the constructor [1, 2].

Circle.execute() delegates to renderCircle(radius), while Square.execute() delegates to renderSquare(side). Neither shape knows a concrete renderer. All six combinations use the same two shape classes and three renderer classes; no combination subclasses or concrete-type branches are needed.

2. Verification and runtime replacement

Check	Actual result / evidence
T1 PASS	VECTOR circle radius=2
T2 PASS	RASTER circle radius=2
T3 PASS	VECTOR square side=3
T4 PASS	RASTER square side=3
T5 PASS	sameObject=true; stateUnchanged=true; VECTOR -> RASTER
T6 PASS	ASCII circle radius=2
T7 PASS	ASCII square side=3

SUMMARY: 7/7 PASS. Compiled with --release 17; executed on Java 17.0.16. T5 compares the saved reference using `original == afterSwitch`. The ID remains circle-switch and radius remains 2. Actual outputs before and after replacement are recorded in demo-output.txt.

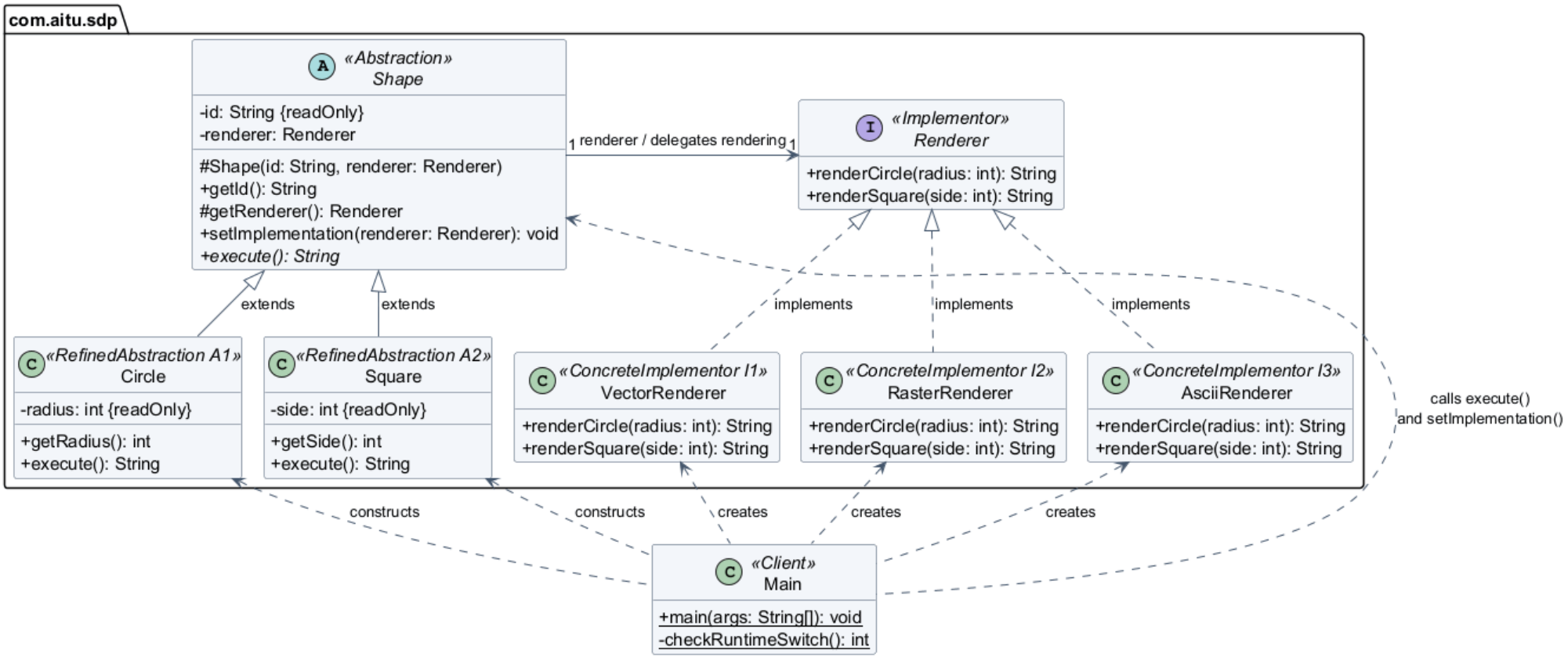
3. Independent extension and comparison

The base commit passed T1-T5 with VectorRenderer and RasterRenderer. The extension added AsciiRenderer and updated only Main.java within src/. Shape, Circle, Square, Renderer and both initial renderers stayed unchanged; sources.txt was updated. extension.diff contains the committed source difference [1].

Trade-off: the extra interface and delegation add indirection. Also, a new shape kind needing a new low-level operation would require updating Renderer and its implementations. **Bridge vs Adapter:** Bridge deliberately separates dimensions that should evolve independently; Adapter translates an existing incompatible interface. This application designs a shared Renderer contract rather than adapting a legacy API [2, 3].

4. UML class diagram

Bridge - Drawing | Main is in the default package; domain classes are in com.aitu.sdp.



Abstraction: Shape stores the ID and an interface-typed Renderer reference. **Refined abstractions:** Circle supplies its radius to renderCircle(); Square supplies its side to renderSquare(). **Implementor:** Renderer defines these low-level operations. VectorRenderer, RasterRenderer and AsciiRenderer implement the same contract with distinct output styles.

Client: Main constructs the shapes and renderers, checks execute() results, and calls setImplementation() for runtime replacement. Only the renderer reference changes; the shape object, its ID and its dimension stay the same. Solid hollow-triangle arrows mean inheritance; dashed hollow-triangle arrows mean interface realization. The solid Shape-to-Renderer association represents object composition with one active renderer. Dashed open arrows show client dependencies. Every visible relation is labeled.

5. Clean Code evidence

5.1 Separation of responsibilities

```
public String execute() {
    return getRenderer().renderCircle(radius);
}
```

src/com/aitu/sdp/Circle.java:16
Circle selects the domain operation; the renderer owns the output style. Benefit: changing a rendering style leaves shape behavior unchanged.

5.2 Meaningful names

```
public String renderSquare(int side) {
    return "RASTER square side=" + side;
}
```

src/com/aitu/sdp/RasterRenderer.java:10
RasterRenderer, renderSquare and side identify the style, operation and dimension. Benefit: the code communicates its purpose without comments.

5.3 Small, focused methods

```
public final void setImplementation(Renderer renderer) {
    this.renderer = Objects.requireNonNull(renderer);
}
```

src/com/aitu/sdp/Shape.java:22
The setter validates and replaces one collaborator. Benefit: replacement is easy to inspect and cannot accidentally modify the ID or dimensions.

5.4 Avoidance of duplicated workflow

```
String actual = shape.execute();
boolean passed = expected.equals(actual);
```

src/Main.java:38
Main.checkResult() shares execution, comparison, output and failure reporting for T1-T4 and T6-T7. Benefit: all six result checks use one consistent workflow.

5.5 Encapsulation of state

```
private final String id;
private Renderer renderer;
```

src/com/aitu/sdp/Shape.java:6
Shape keeps both fields private; ID is final. Circle.radius and Square.side are also private final. Benefit: external code can read domain state but can only replace the renderer through the controlled setter.

6. References

[1] Yerassyl Bekenov. *Assignment 3 | Bridge Pattern*. Astana IT University, ShP-2216 Software Design Patterns, 2026-2027, Sections 2-6.
[2] Refactoring.Guru. *Bridge*. refactoring.guru/design-patterns/bridge.
[3] Refactoring.Guru. *Adapter*. refactoring.guru/design-patterns/adapter.